

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Тема: Коммивояжёр
Вариант 1

Студент гр. 4345

Бабий Д.Н.

Преподаватель

Жангиров Т.Р.

Цель работы

Изучить и реализовать алгоритм Литтла, приближённый алгоритм: 2-приближение

Постановка задачи

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные:

n - количество городов ($5 \leq n \leq 15$).

Матрица расстояний между городами размером $n \times n$, где $graph[i][j]$ обозначает расстояние от города i до города j . Если $graph[i][j]=0$ (и $i \neq j$), это означает, что прямого пути между городами нет.

Выходные данные:

Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.

Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.

Если такого пути не существует, вывести "no path".

Разработайте программу, которая решает задачу коммивояжера при помощи 2-приближенного алгоритма. В данной постановке задачи нужно вернуться в исходную вершину после прохождения всех остальных вершин. При обходе остовного дерева (MST) необходимо идти по минимальному допустимому ребру из текущего. Каждая вершина в графе обозначается неотрицательным числом, начиная с 0, каждое ребро имеет неотрицательный вес. В графе нет рёбер из вершины в саму себя, в матрице весов на месте таких отсутствующих рёбер стоит значение -1.

Пример входных данных

2

-1 18.97 22.36 19.42 3.61

18.97 -1 35.61 38.01 17.0
22.36 35.61 -1 16.28 21.19
19.42 38.01 16.28 -1 21.02
3.61 17.0 21.19 21.02 -1

В первой строке указывается начальная вершина.

Далее идёт матрица весов.

В качестве выходных данных необходимо представить длину пути, полученного при помощи алгоритма. Следующей строкой необходимо представить путь, в котором перечислены вершины, по которым необходимо пройти от начальной вершины. Для приведённых в примере входных данных ответом будет

91.92

2 3 0 4 1 2.

Вариант 1. МВиГ: Алгоритм Литтла. Приближённый алгоритм: 2-приближение по МОД (АДО МОД).Замечание к варианту 1. АДО МОД является 2-приближением только для евклидовой матрицы. Начинать обход МОД со стартовой вершины.

Выполнение работы

1) Алгоритм Литтла

На вход в программу подаются размер квадратной матрицы N и сама матрица. Эта матрица является матрицей расстояний, где значение в $matrix[i][j]$ задаёт стоимость перехода из города i в город j . По главной диагонали значения принимают значения “бесконечности”, так как является петлей.

Реализованные функции:

1.1 copy_matrix(matrix: list[list]) → list[list]

Создаёт полную копию матрицы для независимой обработки ветвей дерева решений.

Пространственная сложность: $O(N^2)$

Временная сложность: $O(N^2)$

2.2 reduce_matrix(matrix: list[list]) → list[list]

Выполняется редукция строк и столбцов:

1. Для каждой строки находится минимальный элемент;
2. Минимум вычитается из всех элементов строки;
3. Аналогично выполняется редукция столбцов.

Сумма всех вычитенных минимумов образует нижнюю границу стоимости текущего решения.

Пространственная сложность: $O(N^2)$

Временная сложность: $O(N^2)$

2.3 find_best_zero(matrix: list[list]) → list[list]

Алгоритм выбирает нулевой элемент матрицы с максимальным штрафом (penalty). Для каждого из них вычисляется минимальный элемент 1) в строке без учёта текущего столбца; 2) в столбце без учёта текущей строки.

Штраф определяется как: $penalty = row_min + col_min$

Выбирается нулевой элемент с максимальным значением штрафа.

Пространственная сложность: $O(N)$

Временная сложность: $O(N^3)$

2.4 `creates_cycle(edges: list, start: int, end: int, n: int) → list[list]`

Функция проверяет, приведёт ли добавление дуги (`start`, `end`) к образованию преждевременного цикла.

2.5 `little(matrix)`

`add_pattern()` — функция построения бора. Процесс начинается с корня и последовательно обрабатывает символы строки. Для каждого символа проверяется наличие соответствующего перехода: если переход отсутствует, создаётся новая вершина, иначе используется уже существующая. За счёт этого одинаковые префиксы различных шаблонов хранятся только один раз. После обработки всей строки конечная вершина помечается как терминальная, а в массив `output` сохраняется идентификатор шаблона. Общая сложность построения дерева составляет $O(\sum m_i)$.

Построение ссылок

Суффиксные ссылки: `build_links()` — аналог префикс-функции в КМП, но для дерева. Для каждой вершины определяется ссылка на наибольший собственный суффикс строки, соответствующей пути к этой вершине, который также является префиксом одного из шаблонов. Если вершина является корнем или её родитель — корень, ссылка ведёт в корень. В противном случае она вычисляется рекурсивно через переходы автомата. Эти ссылки позволяют эффективно “откатываться” при несовпадении, не возвращаясь к началу.

Терминальные ссылки `build_term_links()` — предназначены для быстрого обнаружения всех шаблонов, оканчивающихся в текущей позиции текста. Если вершина, на которую указывает `suffix link`, является терминальной, `terminal link` направляется на неё. Иначе выполняется дальнейший переход по цепочке `terminal link`. Благодаря этому алгоритм способен находить вложенные и пересекающиеся совпадения без дополнительного обхода дерева.

Построение автомата `build()` — выполняется обходом дерева в ширину (BFS) с использованием очереди `deque`. Для вершин первого уровня суффиксные ссылки всегда указывают на корень. Далее для каждой вершины

последовательно вычисляются suffix и terminal ссылки. Построение автомата выполняется за линейное время относительно количества вершин бора.

Поиск в тексте `search()` — текст обрабатывается за один проход слева направо. Текущее состояние автомата хранится в переменной `v`. Для каждого символа текста:

- если соответствующий переход существует, выполняется переход по ребру;
- иначе происходит откат по цепочке suffix link.

Сложность алгоритма:

По времени:

$$O(n + \sum m_i)$$

По памяти:

$$O(\sum m_i)$$

где:

n — длина текста;

m_i — длины шаблонов.

Исходный код программы приведён в приложении А.

Вторая программа сделана для решения задачи наличия wildcard-ов в тексте.

Для поиска шаблона с wildcard-символами была разработана отдельная модификация алгоритма. При прямом сравнении wildcard-шаблона с каждой позицией текста сложность алгоритма может достигать $O(n \cdot m)$, где n — длина текста, а m — длина шаблона. Чтобы избежать повторных сравнений, шаблон предварительно разбивается на блоки без wildcard-символов, после чего все блоки ищутся одновременно при помощи автомата Ахо-Корасик.

Разбиение шаблона на блоки — wildcard-шаблон просматривается посимвольно. Все непрерывные последовательности символов между wildcard-символами выделяются в отдельные блоки. Для каждого блока

дополнительно сохраняется его смещение относительно начала исходного шаблона.

Например, шаблон:

- ab?cd??ef

преобразуется в набор:

- ("ab", 0)
- ("cd", 3)
- ("ef", 7)

WildCard Ахо-Корасик:

По времени:

$$O(n + \sum m_i + k)$$

По памяти:

$$O(n + \sum m_i),$$

где m_i — длины блоков, n -длина текста, k - длина wildcard-шаблона.

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1-2.

Таблица 1 – Тестирование regular.py

№	Входные данные	Выходные данные
1	mississippi	2 1
	3	3 2
	issi	5 1
	ss	6 2
	sip	7 3
2	abababab	1 1
	2	2 2
	aba	3 1
	bab	4 2
		5 1
3	abcdefabc	6 2
	4	1 1
	abc	3 3
	def	4 2
		5 4

	cde ef	
--	-----------	--

Таблица 2 – Тестирование joker.py

№	Входные данные	Выходные данные
1	abacabadaba a\$a \$	1 3 5 7 9
2	aaaa \$\$\$ \$	1 2

Выводы

В работе реализованы две версии алгоритма Aho–Corasick algorithm для строкового поиска. Первая версия выполняет поиск множества шаблонов в тексте за один проход, используя бор, суффиксные и терминальные ссылки, и находит все вхождения без повторных сравнений. Вторая версия расширяет алгоритм для поддержки wildcard-символов: шаблон разбивается на блоки, которые ищутся отдельно, а затем результаты объединяются через массив совпадений. Обе реализации работают за линейное время и эффективно обрабатывают большие строки без избыточных вычислений.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл ac.py

```
class Node:
    _node_id_counter = 0

    def __init__(self, parent=None, char_to_parent=None):
        self.id = Node._node_id_counter
        Node._node_id_counter += 1
        self.son: dict[str, 'Node'] = {}
        self.go: dict[str, 'Node'] = {}
        self.parent: 'Node' = parent
        self.char_to_parent: str = char_to_parent
        self.suff_link: 'Node' = None
        self.up: 'Node' = None
        self.is_leaf: bool = False
        self.leaf_pattern_numbers: list[int] = []

class AhoCorasick:
    def __init__(self):
        Node._node_id_counter = 0
        self.root = Node()
        self.pattern_lengths = {}

    def add_string(self, s, pattern_number):
        cur = self.root
        for char in s:
            if char not in cur.son:
                new_node = Node(parent=cur, char_to_parent=char)
                cur.son[char] = new_node
                print(f"[Бор] Добавлен узел {new_node.id} из {cur.id}
по символу '{char}'")
            else:
                print(f"[Бор] Уже существовал узел {cur.son[char].id}
из {cur.id} по символу '{char}'")
                cur = cur.son[char]
        cur.is_leaf = True
```

```

        cur.leaf_pattern_numbers.append(pattern_number)
        self.pattern_lengths[pattern_number] = len(s)
        print(f"[Бор] Узел {cur.id} помечен как конец строки №
{pattern_number}")

def get_suff_link(self, v: Node):
    if v.suff_link is None:
        if v == self.root or v.parent == self.root:
            v.suff_link = self.root
        else:
            v.suff_link =
self.get_link(self.get_suff_link(v.parent), v.char_to_parent)
        if v != self.root:
            print(f"    [Автомат] Ссылка п: {v.id} ->
{v.suff_link.id}")
    return v.suff_link

def get_link(self, v, c):
    if c not in v.go:
        if c in v.son:
            v.go[c] = v.son[c]
        elif v == self.root:
            v.go[c] = self.root
        else:
            v.go[c] = self.get_link(self.get_suff_link(v), c)
    return v.go[c]

def get_up(self, v):
    if v.up is None:
        sl = self.get_suff_link(v)
        if sl.is_leaf:
            v.up = sl
        elif sl == self.root:
            v.up = self.root
        else:
            v.up = self.get_up(sl)
    if v != self.root:
        print(f"    [Автомат] Ссылка up: {v.id} -> {v.up.id}")
    return v.up

```

```

def process_text(self, text):
    print(f"{f'\nПОИСК В ТЕКСТЕ {text}':^40}")
    results = []
    cur = self.root
    for i in range(len(text)):
        char = text[i]
        next_node = self.get_link(cur, char)
        print(f"Позиция {i}: '{char}' | Переход {cur.id} ->
{next_node.id}")
        cur = next_node

    temp = cur
    while temp != self.root:
        if temp.is_leaf:
            for p_num in temp.leaf_pattern_numbers:
                start_pos = i - self.pattern_lengths[p_num] +
2
                print(f"          Совпадение: Паттерн №{p_num} в
позиции {start_pos}")
                results.append((start_pos, p_num))
            temp = self.get_up(temp)
    return results

def get_max_chains_from_root(self):
    stats = {"max_suff": 0, "max_up": 0}

    def dfs(v):
        if v != self.root:
            curr_suff_len = 0
            temp = v
            while temp != self.root:
                temp = self.get_suff_link(temp)
                curr_suff_len += 1
                stats["max_suff"] = max(stats["max_suff"],
curr_suff_len)

            curr_up_len = 0
            temp = v

```

```

        while temp != self.root:
            temp = self.get_up(temp)
            if temp != self.root:
                curr_up_len += 1
            stats["max_up"] = max(stats["max_up"], curr_up_len)

    for char in sorted(v.son.keys()):
        dfs(v.son[char])

    dfs(self.root)
    return stats["max_suff"], stats["max_up"]

def print_automaton(self):
    print(f"'ИТОГОВАЯ СТРУКТУРА АВТОМАТА':^60}")

    def show(v, char="ROOT", prefix="", is_last=True):
        sl = self.get_suff_link(v)
        up = self.get_up(v)

        leaf_info = f" (TERM: {v.leaf_pattern_numbers})" if
v.is_leaf else ""
        links = f" |  $\pi$ :{sl.id} up:{up.id}" if v != self.root else
""

        connector = "└─ " if is_last else "├─ "
        print(f"{prefix}{connector}'{char}' [ID:{v.id}]{leaf_info}
{links}")

        new_prefix = prefix + ("    " if is_last else "|    ")
        child_chars = sorted(v.son.keys())
        for i, c in enumerate(child_chars):
            show(v.son[c], c, new_prefix, i == len(child_chars) -
1)

    show(self.root)

if __name__ == "__main__":

    text = input()

```

```

n = int(input())
patterns = []
for _ in range(n):
    patterns.append(input())

ac = AhoCorasick()
print(f"{'ПОСТРОЕНИЕ БОРА':^40}")
for i in range(n):
    print(f"Строка '{patterns[i]}'")
    ac.add_string(patterns[i], i+1)

matches = ac.process_text(text)

matches.sort()

for pos, p_idx in matches:
    print(f"{pos} {p_idx}")

suff_max, up_max = ac.get_max_chains_from_root()
print("Максимальная цепочка суффиксных ссылок " + str(suff_max))
print("Максимальная цепочка конечных ссылок " + str(up_max))

ac.print_automaton()

```

Файл ac_wc.py

```

class Node:
    _node_id_counter = 0

    def __init__(self, parent=None, char_to_parent=None):
        self.id = Node._node_id_counter
        Node._node_id_counter += 1
        self.son: dict[str, 'Node'] = {}
        self.go: dict[str, 'Node'] = {}
        self.parent: 'Node' = parent
        self.char_to_parent: str = char_to_parent
        self.suff_link: 'Node' = None
        self.up: 'Node' = None
        self.is_leaf: bool = False
        self.leaf_pattern_numbers: list[int] = []

```

```

class AhoCorasickWildCard:
    def __init__(self):
        Node._node_id_counter = 0
        self.root = Node()
        self.pattern_lengths = {}

    def add_string(self, s, pattern_number):
        cur = self.root
        for char in s:
            if char not in cur.son:
                new_node = Node(parent=cur, char_to_parent=char)
                cur.son[char] = new_node
                print(f"[Бор] Добавлен узел {new_node.id} из {cur.id}
по символу '{char}'")
            else:
                print(f"[Бор] Уже существовал узел {cur.son[char].id}
из {cur.id} по символу '{char}'")
                cur = cur.son[char]
            cur.is_leaf = True
            cur.leaf_pattern_numbers.append(pattern_number)
            self.pattern_lengths[pattern_number] = len(s)
            print(f"[Бор] Узел {cur.id} помечен как конец строки №
{pattern_number}")

    def get_suff_link(self, v: Node):
        if v.suff_link is None:
            if v == self.root or v.parent == self.root:
                v.suff_link = self.root
            else:
                v.suff_link =
self.get_link(self.get_suff_link(v.parent), v.char_to_parent)
            if v != self.root:
                print(f"    [Автомат] Ссылка п: {v.id} ->
{v.suff_link.id}")
        return v.suff_link

    def get_link(self, v, c):
        if c not in v.go:

```



```

        if c in v.son:
            v.go[c] = v.son[c]
        elif v == self.root:
            v.go[c] = self.root
        else:
            v.go[c] = self.get_link(self.get_suff_link(v), c)
    return v.go[c]

def get_up(self, v):
    if v.up is None:
        sl = self.get_suff_link(v)
        if sl.is_leaf:
            v.up = sl
        elif sl == self.root:
            v.up = self.root
        else:
            v.up = self.get_up(sl)
    if v != self.root:
        print(f" [АВТОМАТ] Ссылка: {v.id} -> {v.up.id}")
    return v.up

def process_text_with_wildcards(self, text, parts,
full_pattern_len):
    print(f"{f'\nПОИСК В ТЕКСТЕ {text}':^40}")
    counts = [0] * len(text)
    cur = self.root

    for i in range(len(text)):
        char = text[i]
        next_node = self.get_link(cur, char)
        print(f"Позиция {i}: '{char}' | {cur.id} ->
{next_node.id}")
        cur = next_node

    temp = cur
    while temp != self.root:
        if temp.is_leaf:
            for p_idx in temp.leaf_pattern_numbers:
                offset = parts[p_idx][1]

```

```

        start_pos = i - self.pattern_lengths[p_idx] -
offset + 1

        print(f"        Совпадение: часть №{p_idx},
ВОЗМОЖНЫЙ старт = {start_pos}")

        if 0 <= start_pos <= len(text) -
full_pattern_len:

            counts[start_pos] += 1
            print(f"                Засчитано в
counts[{start_pos}] = {counts[start_pos]}")

            temp = self.get_up(temp)

        return counts

def print_automaton(self):
    print(f"{'\nИТОГОВАЯ СТРУКТУРА АВТОМАТА':^60}")

def show(v, char="ROOT", prefix="", is_last=True):
    sl = self.get_suff_link(v)
    up = self.get_up(v)

    leaf_info = f" (TERM: {v.leaf_pattern_numbers})" if
v.is_leaf else ""
    links = f" | π:{sl.id} up:{up.id}" if v != self.root else
""

    connector = "└─ " if is_last else "├─ "
    print(f"{prefix}{connector}'{char}' [ID:{v.id}]{leaf_info}
{links}")

    new_prefix = prefix + ("  " if is_last else "|  ")
    child_chars = sorted(v.son.keys())
    for i, c in enumerate(child_chars):
        show(v.son[c], c, new_prefix, i == len(child_chars) -
1)

    show(self.root)

```

```

if __name__ == "__main__":
    text = input()
    pattern = input()
    wildcard = input()

    parts = []
    start = -1

    print(f"{'\nРАЗБИЕНИЕ ШАБЛОНА':^40}")
    for i in range(len(pattern)):
        if pattern[i] != wildcard:
            if start == -1:
                start = i
            else:
                if start != -1:
                    parts.append((pattern[start:i], start))
                    print(f"Часть: '{pattern[start:i]}' с offset {start}")
                    start = -1

    if start != -1:
        parts.append((pattern[start:], start))
        print(f"Часть: '{pattern[start:]}' с offset {start}")

    ac = AhoCorasickWildCard()

    print(f"{'\nПОСТРОЕНИЕ БОРА':^40}")
    for i in range(len(parts)):
        print(f"Строка '{parts[i][0]}'")
        ac.add_string(parts[i][0], i)

    counts = ac.process_text_with_wildcards(text, parts, len(pattern))

    print(f"{'\nПРОВЕРКА СОВПАДЕНИЙ':^40}")
    num_parts = len(parts)
    for i in range(len(counts)):
        if counts[i] == num_parts:
            print(f"Совпадение полного шаблона с позиции {i + 1}")

```

```
ac.print_automaton()
```